



UNICAMP

UNIVERSIDADE ESTADUAL DE CAMPINAS

Faculdade de Tecnologia

Sistema Web para Avaliação de Disciplinas

Documentação Técnica do Sistema

Victor Valentim Bergamasco

Orientador: Prof. Dr. Vitor Rafael Coluci

Limeira – Junho de 2026

1 Apresentação

Este documento descreve, em nível técnico, como está organizado o sistema web de avaliação de disciplinas desenvolvido como Trabalho de Conclusão de Curso. A leitura é direcionada a quem precisa entender ou continuar o código — outro estudante, um professor, ou eu mesmo daqui a alguns meses.

A monografia traz a fundamentação e a motivação. Aqui, o foco é prático: qual pasta contém o quê, quais decisões de implementação foram tomadas e onde encontrar cada parte importante do código. Quando faz sentido mostrar código, o trecho aparece em forma de figura, com a indentação e o destaque de sintaxe preservados, como apareceria em um editor.

2 Como o projeto está organizado

Tudo vive em duas pastas: *backend_tcc* e *frontend_tcc*. Cada uma roda de forma independente, em portas diferentes, e conversa com a outra exclusivamente por HTTP. Na prática isso significa que o time do back-end pode mexer no Django sem precisar tocar no JavaScript, e vice-versa.

Para não ter que abrir dois terminais sempre que quiser usar o sistema, deixei um script chamado *iniciar-sistema.bat* na raiz do projeto. Um duplo clique nele sobe os dois servidores em janelas separadas e ainda abre o navegador na URL do front-end. Útil também para apresentações.

Dentro de *backend_tcc* existem cinco aplicações Django, cada uma responsável por um pedaço bem delimitado: *users* cuida de autenticação e usuários; *courses* guarda as disciplinas e gera os QR Codes; *evaluations* armazena as perguntas e as respostas anônimas; *dashboard* calcula totais e médias; e *reports* monta o PDF do relatório que o professor baixa.

3 Back-end (Django)

3.1 Aplicação users — login, papéis e primeiro acesso

O modelo de usuário foi customizado para autenticar por e-mail institucional. Em vez do *username* padrão do Django, o login do sistema é o próprio e-mail, e cada usuário carrega um campo *role* que vale “admin” ou “professor”. Há também uma flag chamada *must_change_password*, que é a chave de todo o fluxo de primeiro acesso: o administrador cria o professor com uma senha provisória e essa flag começa em *True*, forçando o professor a definir uma senha pessoal antes de poder usar qualquer outra tela do sistema.

```

class User(AbstractUser):
    ROLE_CHOICES = (
        ("admin", "Administrador"),
        ("professor", "Professor"),
    )

    username = None
    email = None

    institutional_email = models.EmailField(unique=True)
    role = models.CharField(
        max_length=20, choices=ROLE_CHOICES, default="professor"
    )
    must_change_password = models.BooleanField(default=False)

    USERNAME_FIELD = "institutional_email"
    REQUIRED_FIELDS = ["first_name", "last_name"]

    objects = UserManager()

```

Figura 1 — Modelo User customizado (*apps/users/models.py*).

As regras de acesso ficam em *permissions.py*, em duas classes muito pequenas e expressivas. *IsAdminRole* exige que o usuário esteja autenticado e seja admin. *IsProfessorOrAdmin* aceita qualquer um dos dois papéis. Cada endpoint da API declara qual classe quer usar, e o DRF resolve isso antes mesmo do código da view rodar.

3.2 Aplicação courses — QR Code e perguntas automáticos

A criação de uma disciplina dispara dois efeitos colaterais que poderiam ser feitos no nível das views, mas que eu preferi colocar diretamente no *save()* do modelo: a geração do QR Code e a replicação das vinte e quatro perguntas-padrão da FT. Centralizar isso no modelo garante que, não importa de onde a disciplina seja criada (interface web, shell do Django, importação em lote), as automações vão acontecer.

```

def save(self, *args, **kwargs):
    creating = self._state.adding

    should_generate = False
    if self.google_form_link:
        if not self.pk or not self.qr_code_image:
            should_generate = True
        else:
            old = Course.objects.filter(pk=self.pk).first()
            if old and old.google_form_link != self.google_form_link:
                should_generate = True

    if should_generate:
        self.generate_qr_code()

    super().save(*args, **kwargs)

    if creating:
        from apps.evaluations.standard_questions import sync_standard_questions
        sync_standard_questions(self)

```

Figura 2 — Sobrecarga do `save()` do modelo `Course`.

A função `generate_qr_code()` usa a biblioteca `qrcode` para construir a imagem em memória e entregá-la ao `ImageField` do Django, que se encarrega de gravar o PNG no disco. O nome do arquivo é derivado do código da disciplina, o que evita conflitos entre cursos diferentes e facilita a identificação manual quando preciso depurar.

```

def generate_qr_code(self):
    qr = qrcode.make(self.google_form_link)
    buffer = BytesIO()
    qr.save(buffer, format="PNG")
    filename = f"qr_course_{self.code}.png"
    self.qr_code_image.save(filename, File(buffer), save=False)

```

Figura 3 — Geração da imagem do QR Code.

3.3 Aplicação `evaluations` — perguntas, respostas e anonimato

Esta é a parte central. O modelo `StandardQuestion` guarda a lista oficial das vinte e quatro perguntas da FT. Cada nova disciplina recebe uma cópia desses registros como `EvaluationQuestion` associadas a ela. Manter duas entidades distintas para template e instâncias permite editar o texto de uma pergunta na disciplina sem afetar as outras, e ao mesmo tempo dá ao administrador o controle central do template.

Para garantir o anonimato dos respondentes, a entidade `EvaluationResponse` — que representa cada submissão do formulário — foi deliberadamente desenhada sem nenhuma referência a usuário. Não guarda IP, não guarda sessão, não tem `ForeignKey` para `User`. Mesmo com acesso direto ao banco, não dá para descobrir quem respondeu o quê. É uma propriedade do esquema, não uma promessa operacional.

A função que faz a replicação das perguntas para uma disciplina foi escrita de forma idempotente: pode ser chamada várias vezes sobre o mesmo curso que ela só cria o que estiver

faltando. Isso é essencial para o botão “Aplicar em todas as disciplinas” da tela administrativa de perguntas-padrão funcionar sem risco de duplicação.

```
def sync_standard_questions(course, *, overwrite_text=False):
    from apps.evaluations.models import EvaluationQuestion

    template = _get_template_questions()
    criadas = atualizadas = 0

    for q in template:
        existing = EvaluationQuestion.objects.filter(
            course=course, order=q["order"]
        ).first()
        if existing is None:
            EvaluationQuestion.objects.create(
                course=course, order=q["order"],
                text=q["text"], question_type=q["question_type"],
            )
            criadas += 1
        elif overwrite_text:
            existing.text = q["text"]
            existing.question_type = q["question_type"]
            existing.save(update_fields=["text", "question_type"])
            atualizadas += 1

    return criadas, atualizadas
```

Figura 4 — Sincronização idempotente do template de perguntas.

3.4 Aplicação dashboard — estatísticas

O dashboard tem dois endpoints. O primeiro é a visão geral do administrador, com três números: quantos professores, quantas disciplinas e quantas submissões existem. Uma decisão pequena mas importante: o total de respostas conta submissões inteiras (*EvaluationResponse*), não a soma de respostas individuais (*EvaluationAnswer*). Antes eu contava o segundo e os números ficavam inflados — um aluno respondendo o formulário gerava 24 no contador.

O segundo endpoint é o que alimenta o gráfico de barras na tela de relatórios. Recebe o id da disciplina e devolve a média de cada pergunta de escala. Aqui entra um detalhe importante de segurança: além da permissão por papel, há uma verificação explícita de que o professor logado é o responsável pela disciplina. Sem isso, qualquer professor poderia trocar o id na URL e ver dados alheios.

```

class CourseDashboard(APIView):
    permission_classes = [IsProfessorOrAdmin]

    def get(self, request, course_id):
        course = get_object_or_404(Course, id=course_id)

        user = request.user
        if user.role == "professor" and course.professor_id != user.id:
            raise PermissionDenied(
                "Você não tem permissão para esta disciplina."
            )

        questions = EvaluationQuestion.objects.filter(
            course=course).order_by("order")

        result = []
        for q in questions:
            answers = EvaluationAnswer.objects.filter(
                question=q, scale_value__isnull=False)
            avg = answers.aggregate(avg=Avg("scale_value"))["avg"]
            result.append({
                "question": q.text,
                "average": round(avg, 2) if avg else 0,
                "total_answers": answers.count(),
            })

        return Response({"course": course.name, "questions": result})

```

Figura 5 — Endpoint do dashboard por disciplina.

3.5 Aplicação reports — geração do PDF

O relatório PDF é montado por inteiro no servidor, em um módulo chamado *services.py*. Eu combinei duas bibliotecas: a *matplotlib* renderiza um histograma para cada pergunta de escala e salva como imagem temporária; a *reportlab* monta o documento, encadeando cabeçalho institucional, tabela de dados da disciplina, os histogramas com legenda e os comentários abertos.

O cálculo estatístico tem duas particularidades. A primeira é que a opção “Não sei avaliar” (valor 0) é desconsiderada no cálculo da média e do desvio padrão — incluí-la enviesaria os resultados, já que ela representa ausência de opinião, não opinião neutra. A segunda é o cálculo do tamanho amostral mínimo, com a fórmula clássica para populações finitas. O resultado aparece no cabeçalho de cada relatório como um sinal de quão confiáveis são os números.

```

def calculate_mean_sd(values):
    # Ignora "Não sei avaliar" (valor 0) no cálculo
    valid = [v for v in values if v > 0]
    if not valid:
        return None, None

    mean = sum(valid) / len(valid)
    variance = sum((x - mean) ** 2 for x in valid) / len(valid)
    return mean, math.sqrt(variance)

def tamanho_amostrai(matriculados, tipo="professor"):
    n_pop = int(matriculados) if matriculados else 0
    if n_pop <= 0:
        return 0

    z = 1.96 # 95% de confiança
    p = 0.5 # variância máxima
    e = 0.15 if tipo == "professor" else 0.05

    aa = (z * z * p * (1 - p)) / (e * e)
    return aa / (1 + (aa / n_pop))

```

Figura 6 — Cálculo estatístico do relatório.

4 Front-end (React)

O front-end foi construído em React com Vite. O ponto principal a destacar é o módulo `services/api.js`, que centraliza toda a comunicação com a API. Em vez de cada tela carregar a URL base e o token manualmente, esse módulo configura uma instância de Axios e instala um *interceptor* que injeta o cabeçalho de autenticação em toda requisição. As telas só importam `api` e usam.

```

import axios from "axios";

const api = axios.create({
  baseURL: "http://127.0.0.1:8000/api/",
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export function getMediaUrl(pathOrUrl) {
  if (!pathOrUrl) return "";
  if (pathOrUrl.startsWith("http")) return pathOrUrl;
  return `http://127.0.0.1:8000${pathOrUrl}`;
}

export default api;

```

Figura 7 — Cliente HTTP centralizado (services/api.js).

A proteção de rotas é feita pelo componente *PrivateRoute*, definido no *App.jsx*. Ele faz duas verificações: se não há token, manda para o login; se há token mas a flag de primeiro acesso ainda está ativa, manda para a tela de troca de senha. Esse duplo controle garante que o sistema nunca seja operado com a senha provisória que o administrador definiu.

```

function PrivateRoute({ children }) {
  if (!localStorage.getItem("token")) {
    return <Navigate to="/" replace />;
  }
  try {
    const user = JSON.parse(localStorage.getItem("user") || "{}");
    if (user.must_change_password) {
      return <Navigate to="/trocar-senha" replace />;
    }
  } catch (e) { /* ignore */ }
  return children;
}

```

Figura 8 — Proteção de rotas e bloqueio de primeiro acesso.

5 Fluxos principais

5.1 Primeiro login do professor

Quando o administrador cadastra um professor, ele define uma senha provisória e o sistema marca o campo *must_change_password* como verdadeiro. No primeiro login, o front-end consulta o endpoint */api/users/me/* logo depois de receber o token, percebe a flag ligada e redireciona o professor para */trocar-senha*. A tela de troca não pede a senha atual nesse caso — só a nova. Ao salvar, o back-end aplica o hash, desliga a flag e o professor segue para o dashboard normalmente. Da segunda vez em diante, qualquer troca de senha pelo próprio professor passa a exigir a senha atual.

5.2 Recuperação de senha por e-mail

Quem clica em “Esqueceu sua senha?” na tela de login informa apenas o e-mail. O servidor verifica se existe um usuário ativo com aquele e-mail; se sim, gera um token único (com o *PasswordResetTokenGenerator* do Django, vinculado ao hash atual da senha) e envia o link de redefinição por e-mail. A resposta para o front-end é sempre a mesma mensagem genérica — “se o e-mail estiver cadastrado, enviamos um link” — para não revelar quais e-mails fazem parte da base.

Em ambiente de desenvolvimento, o e-mail não vai para o Gmail nem nada parecido: o Django está configurado com o *console.EmailBackend*, então o e-mail completo (com o link de reset) aparece impresso no terminal onde o *runserver* está rodando. Em produção basta trocar para o backend SMTP e preencher as credenciais no *.env*.

5.3 Submissão anônima da avaliação

Esta é a parte mais simples do sistema e a mais importante para garantir o anonimato. O aluno escaneia o QR Code da disciplina; o navegador abre uma rota pública que mostra as vinte e quatro perguntas; o aluno preenche e envia. O endpoint que recebe a submissão é marcado como *AllowAny* no DRF e cria um registro de *EvaluationResponse* que, conforme já mencionado, não tem qualquer vínculo com a identidade do respondente.

6 Banco de dados

O esquema tem cinco tabelas principais. *users_user* guarda administradores e professores. *courses_course* guarda as disciplinas com referência ao professor responsável. *evaluations_standardquestion* guarda o template de perguntas. *evaluations_evaluationquestion* guarda as cópias específicas de cada disciplina. *evaluations_evaluationresponse* e *evaluations_evaluationanswer* guardam as submissões dos alunos e as respostas individuais.

Todas as ForeignKeys usam *on_delete=CASCADE*. Apagar uma disciplina apaga junto as perguntas, as submissões e as respostas associadas a ela. Apagar um professor apaga as disciplinas dele. É um comportamento agressivo, mas faz sentido neste contexto: não interessa guardar respostas órfãs de uma disciplina que não existe mais.

O ponto-chave do esquema, do ponto de vista de privacidade, é que *evaluations_evaluationresponse* não tem ForeignKey para *users_user* nem armazena IP ou token de sessão. A tabela tem exatamente três colunas: id, *course_id* (referência à disciplina) e *submitted_at* (data e hora). Nada mais. É essa escolha estrutural que sustenta o anonimato; o resto é consequência.

7 Endpoints da API

A tabela abaixo lista os endpoints principais. Todas as rotas estão sob o prefixo `/api/`. O cabeçalho `Authorization: Bearer <token>` é obrigatório em todas, exceto as marcadas como públicas.

Método	Rota	Acesso	Descrição
POST	<code>/auth/login/</code>	público	Autentica e devolve JWT
GET	<code>/users/me/</code>	logado	Dados do usuário atual
POST	<code>/users/me/change-password/</code>	logado	Troca a senha
POST	<code>/users/password-reset/</code>	público	Solicita link de reset
POST	<code>/users/password-reset/confirm/</code>	público	Confirma reset
GET	<code>/users/professores/</code>	admin	Lista professores
POST	<code>/users/professores/</code>	admin	Cria professor
GET	<code>/courses/</code>	logado	Lista disciplinas
POST	<code>/courses/</code>	admin	Cria disciplina
GET	<code>/evaluations/questions/</code>	público	Perguntas do formulário
POST	<code>/evaluations/submit/</code>	público	Submissão anônima
GET	<code>/dashboard/overview/</code>	admin	Estatísticas globais
GET	<code>/dashboard/course/<id>/</code>	prof/admin	Médias por pergunta
GET	<code>/reports/course/<id>/pdf/</code>	prof/admin	Download do PDF

8 Segurança e anonimato

A segurança do sistema é construída em camadas. Da mais externa para a mais interna: o login emite um JWT que precisa estar presente em toda requisição autenticada; as senhas são guardadas em hash com PBKDF2 (padrão do Django); o Django aplica validadores para impedir senhas curtas, comuns ou totalmente numéricas; as permissões por papel filtram quem pode chamar cada endpoint; e, por fim, dentro dos endpoints que retornam dados de disciplinas, há um filtro adicional no *queryset* que limita o professor a ver apenas as próprias disciplinas.

Esse último ponto é o que protege contra o cenário mais óbvio de abuso: um professor tentar consultar dados de outra disciplina trocando o id na URL. A permissão sozinha (*IsProfessorOrAdmin*) deixaria passar; o filtro no *queryset* é quem fecha a porta.

O anonimato dos alunos, como já mencionado, está garantido estruturalmente. O modelo de dados foi desenhado para ser fisicamente incapaz de associar uma resposta a um respondente, independentemente do código de aplicação que rodar sobre ele. É essa propriedade do esquema, e não uma promessa do servidor, que sustenta a confiança do instrumento.

9 Próximos passos

Há algumas frentes que ficaram fora do escopo do TCC mas que valeriam atenção em uma evolução do sistema. A primeira é a produção de testes automatizados — hoje todos os cenários foram validados manualmente, o que dificulta a manutenção. Uma suíte com *pytest* no back-end e *Vitest* no front-end cobrindo os fluxos críticos seria o investimento mais útil.

A segunda frente é a hospedagem em produção. O caminho mais direto é Render para o back-end e o banco, Vercel para o front. Os ajustes necessários estão listados no README do repositório.

A terceira é a internacionalização. Hoje o sistema é integralmente em português brasileiro. Adicionar uma camada de i18n abriria a possibilidade de adoção em outras unidades da universidade ou em instituições parceiras.

Por fim, uma integração com o sistema acadêmico oficial da FT-UNICAMP eliminaria a etapa manual de cadastro de disciplinas a cada semestre, fechando o último ponto operacional repetitivo do ciclo. Esses pontos estão registrados em maior detalhe no Capítulo 9 da monografia.